

Machine Learning Based Design of an Electronic Scarecrow for Protecting Agricultural Yields(Rice) against Predators



Noah Adasi

**Supervisor: Dr. Hyder ALi
Segu Mohammed**



Outline



1 Introduction

2 The Problem

3 Project Objectives

4 System Design and
Implementation



ELECTRICAL , SOFTWARE & MECHANICAL

5 Analysis, Testing & Results

6 Conclusion, Limitations
& Future works





Introduction



Introduction



1 Economic Losses

Predators, particularly birds, pose a significant threat to agricultural production in Africa, causing substantial economic losses and jeopardizing food security for local communities.



2 Inadequate Traditional Methods

Traditional predator deterrence methods, such as scarecrows, often lack adaptability and responsiveness, relying on static stimuli and failing to adjust based on real-time predator behavior.

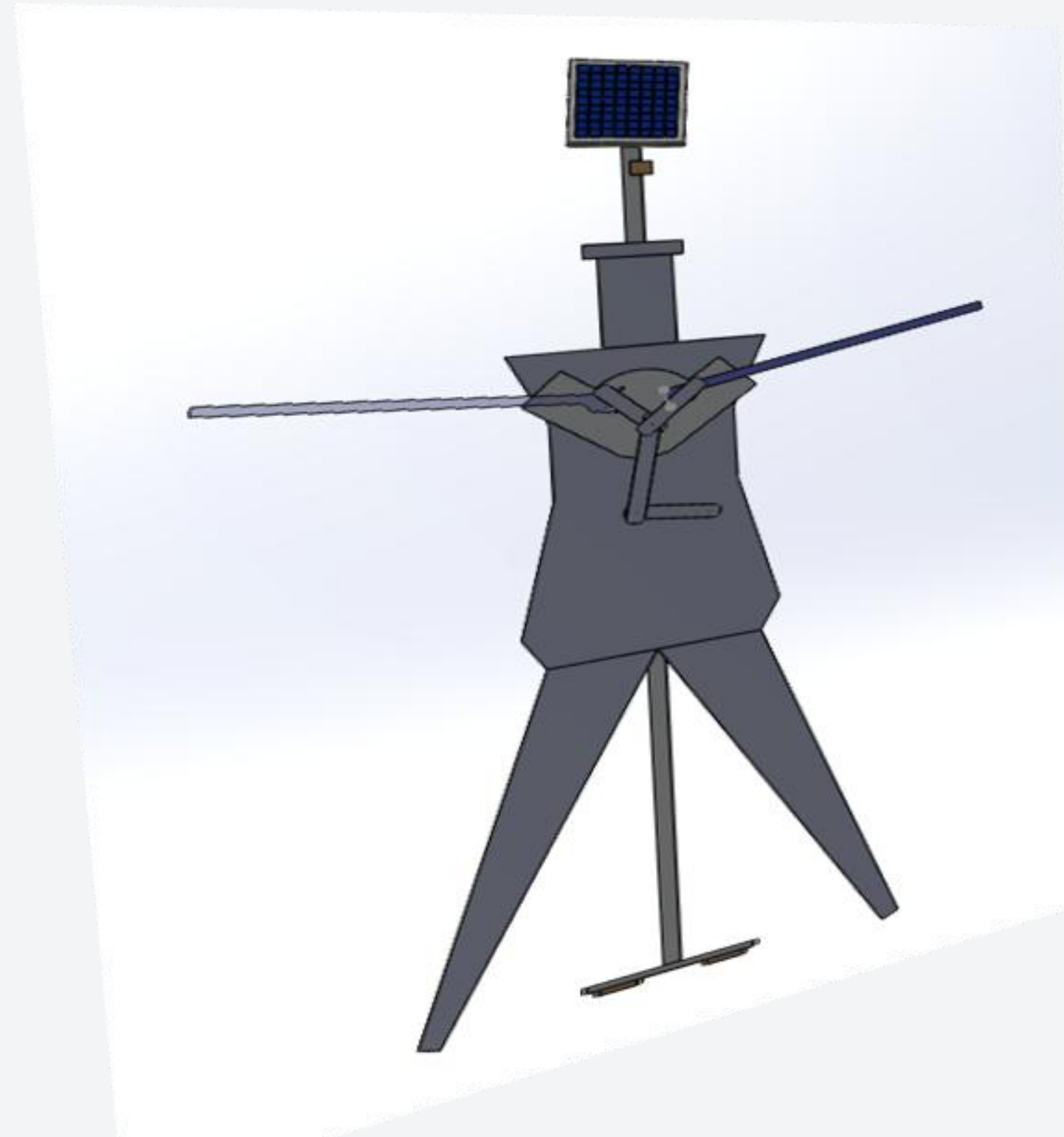


😊 From research:

- Physical chasing, shouting, and scaring off
- Beating sonorous bodies like tins and jerry cans
- Poisoning and trapping
- Use of stationary scarecrows

Need for Dynamic Solution

The "Shield Your Farm" project aims to address these limitations through the development of an intelligent electronic scarecrow that integrates machine learning technologies.



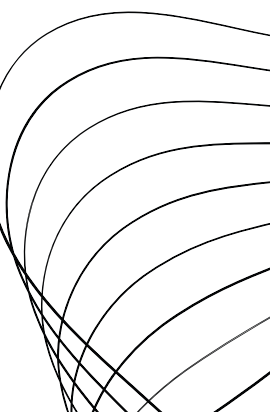
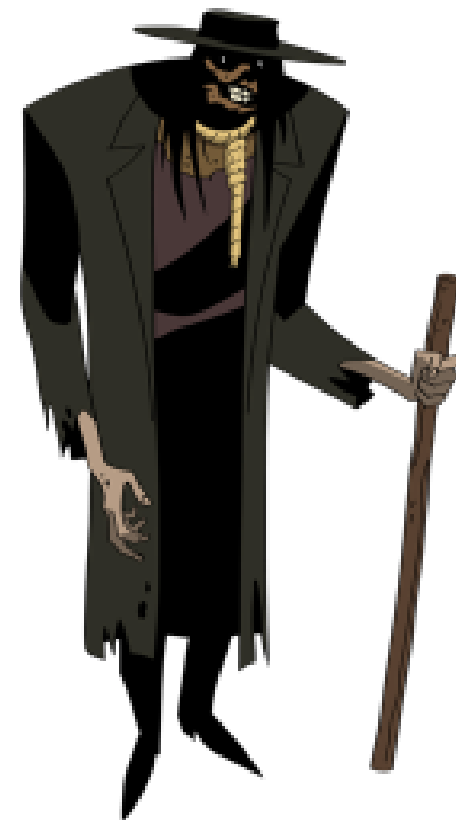
Project Objectives

Machine Learning System Development

Develop and implement machine learning algorithms capable of analyzing real-time camera feed to detect potential bird on rice farms.

Responsive Bird-Scaring Mechanisms

Design and integrate responsive bird-scaring mechanisms, such as sound from speakers, two-dimensional movement mechanisms, and a traditional way of scaring birds, triggered an action when the machine learning model detect birds in the rice farm..



System Design and Implementation

LARANA, INC.



Electrical , Software & Mechanical



Design Requirements

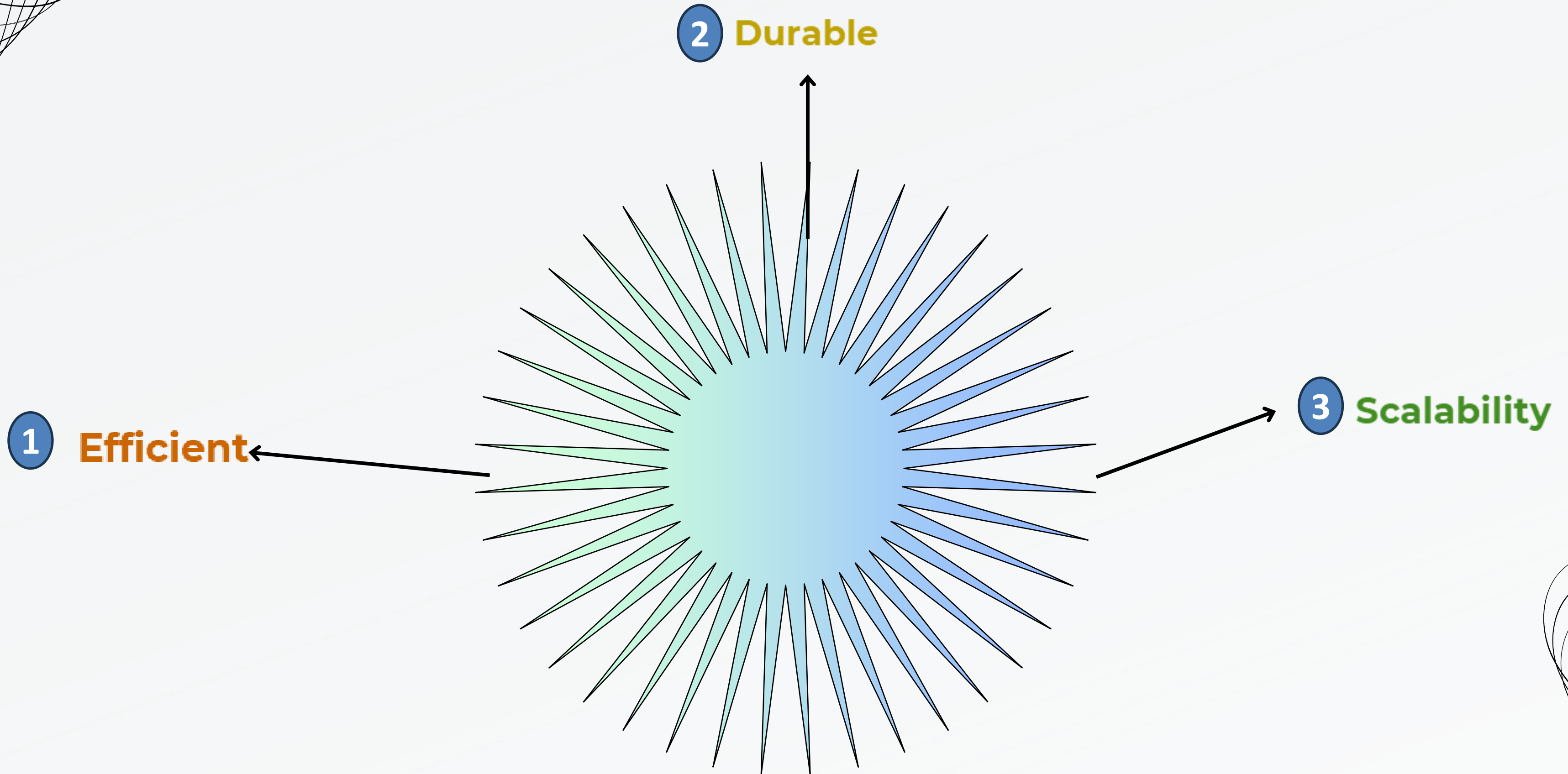
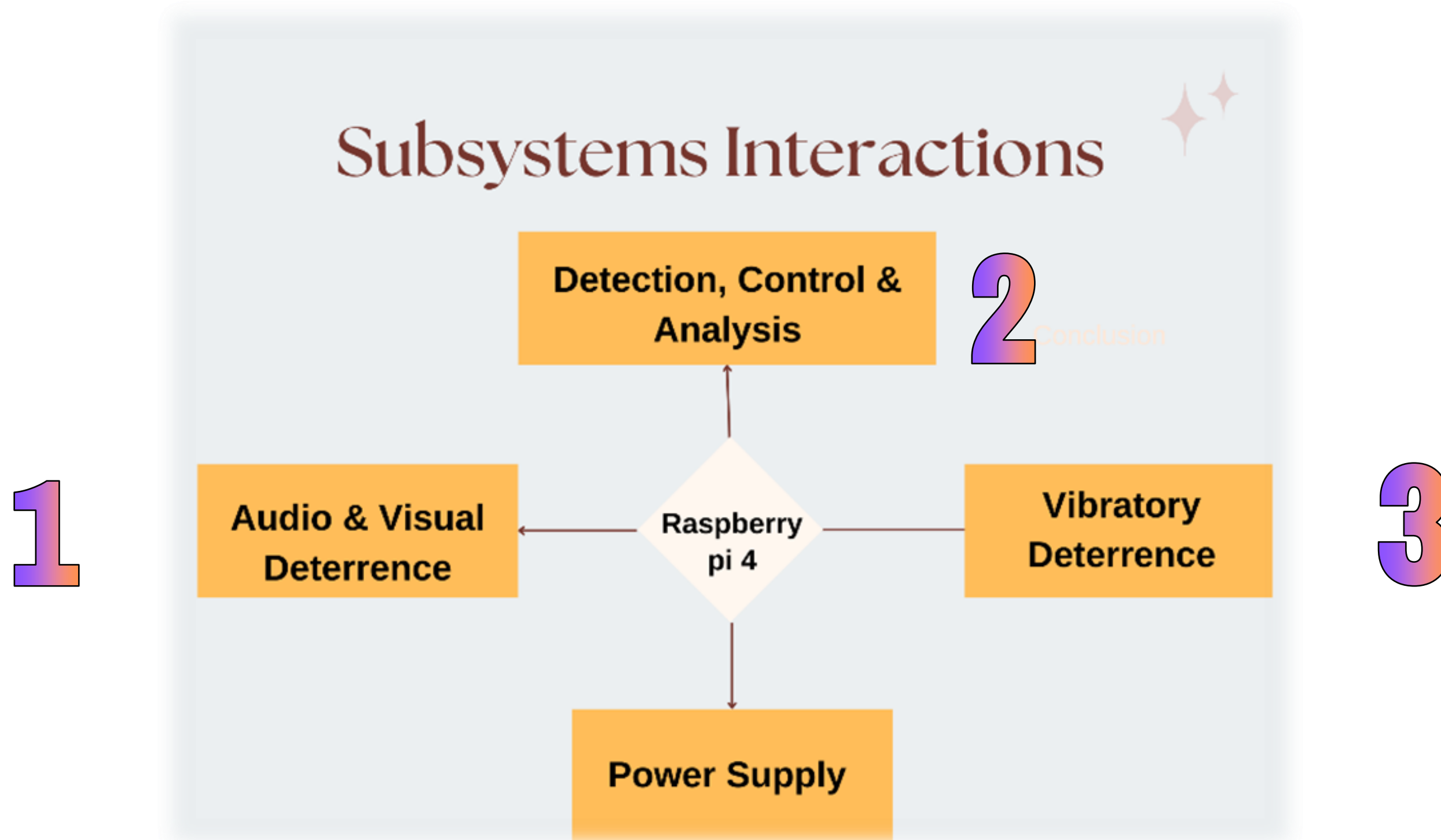


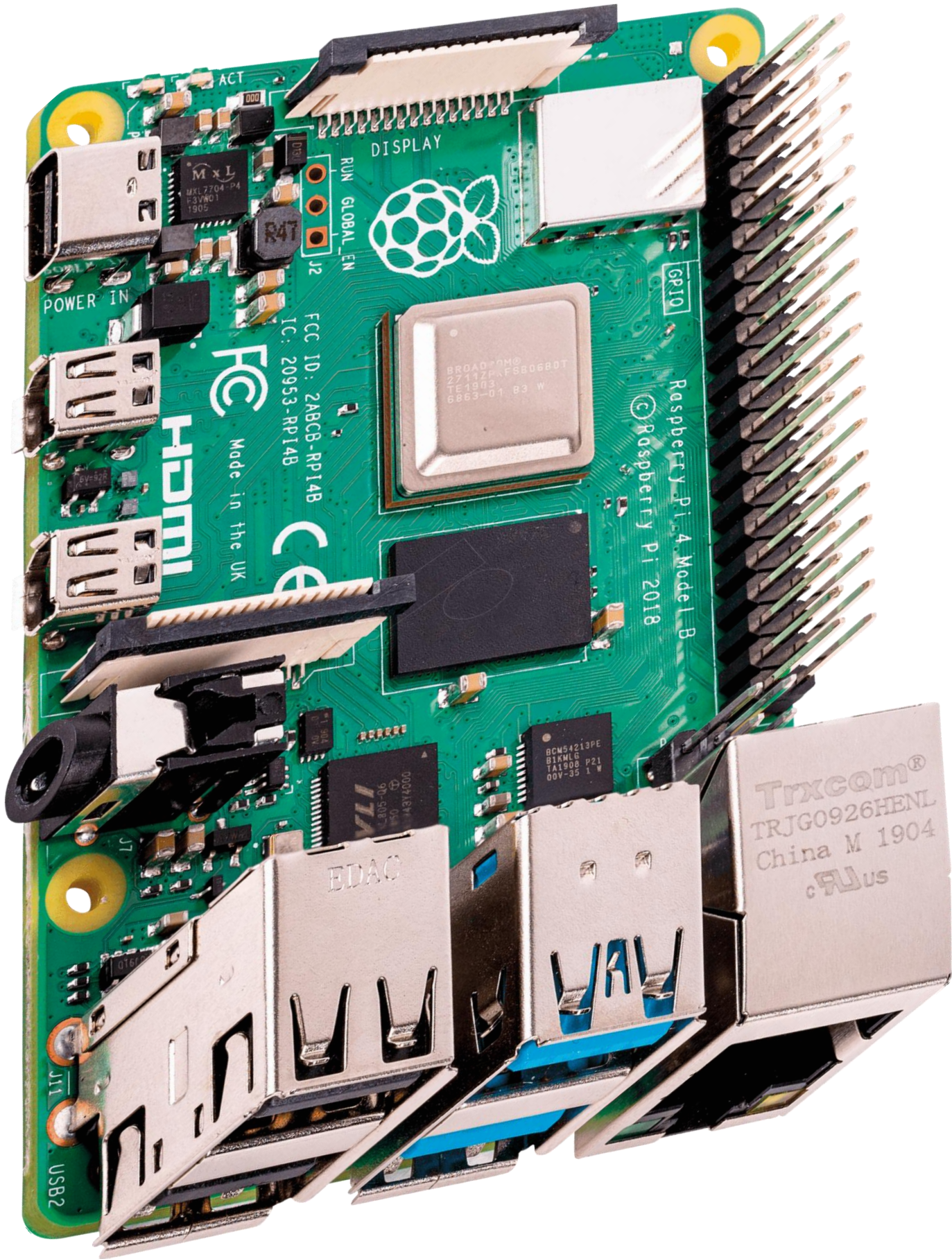
Diagram of Subsystems



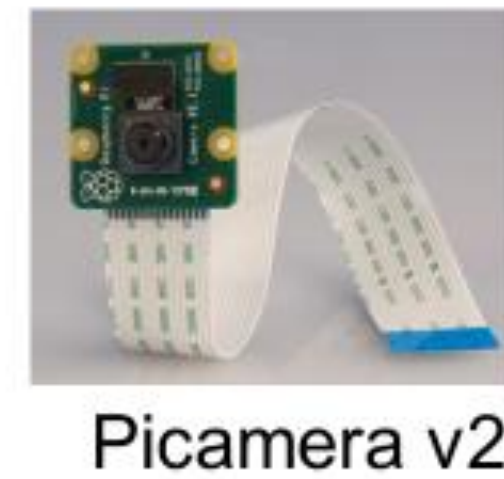
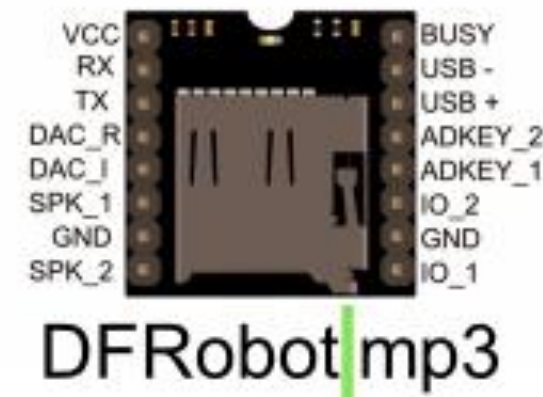
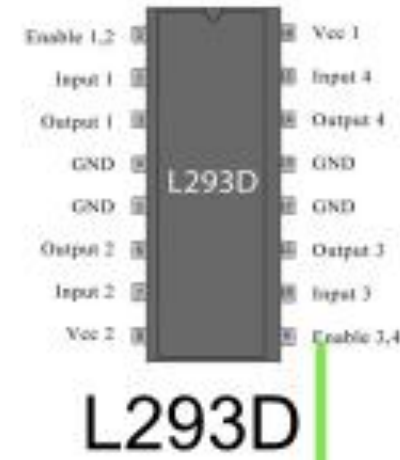
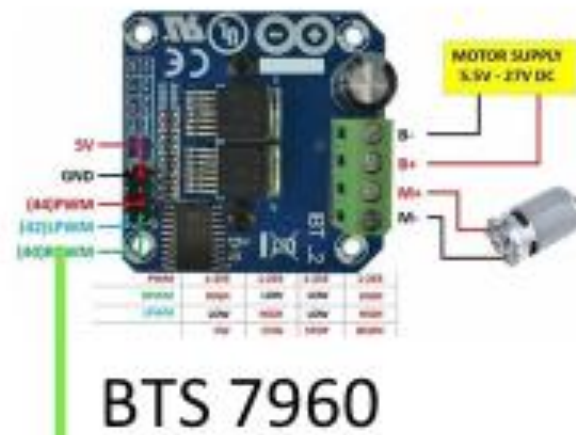
Microcontroller Selection

Criteria	Raspberry Pi 4 B (baseline)	Weight (Out of 5)	ESP32 Cam	Arduino Uno	STM32f1	Node MCU (ESP8266)
High processing power	0	3	-2	-3	-3	-2
Speed	0	2	-1	-3	-3	-2
Availability	0	1	+1	+2	-2	+1
Cost	0	1	+1	+1	-2	+2
Voltage	0	1	+1	+2	+1	+1
TOTAL	0	8	-5	-10	-18	-6

The Raspberry Pi 4 was chosen as the central MCU due to its robust processing capabilities and connectivity options, making it an excellent choice for serving as the project's backbone. Its ability to handle complex tasks and manage multiple connections lays a solid foundation for future expansion and integration of additional functionalities.



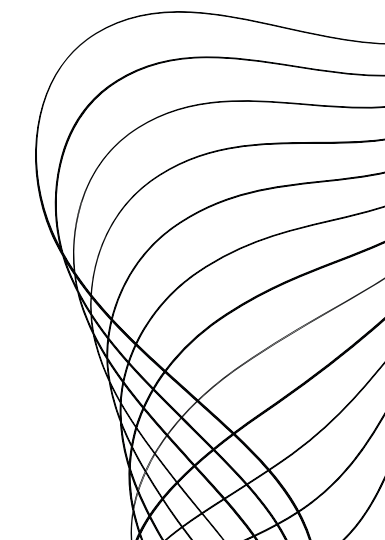
Design – Electrical Components Selection

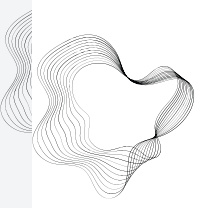




How reliable is the power source to the system?

Components	Current Ratings (A)	Voltage Ratings (V)
MG995 Servo Motor (x2)	1.2	6
Raspberry 4 B	3.00	5.00
775 Motor	1.2	12.00
L298N Motor Driver	0.036	3.2
L293D Motor Driver	0.6	5





How reliable is the power source to the system?

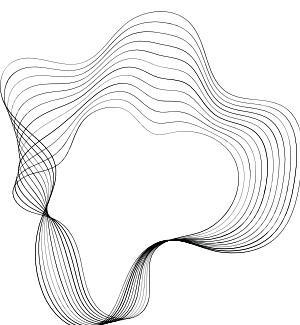
LARANA, INC.

$$PT = [14.4 + 15 + 14.4 + 0.1152 + 3] W$$

$$PT = 46.9152W$$

Battery Specifications:

- ❖ *Voltage* = 12V
- ❖ *Capacity* = 17Ah
- ❖ Power: 204 watts-hour



The Power Source



Power Source



Solar Panel



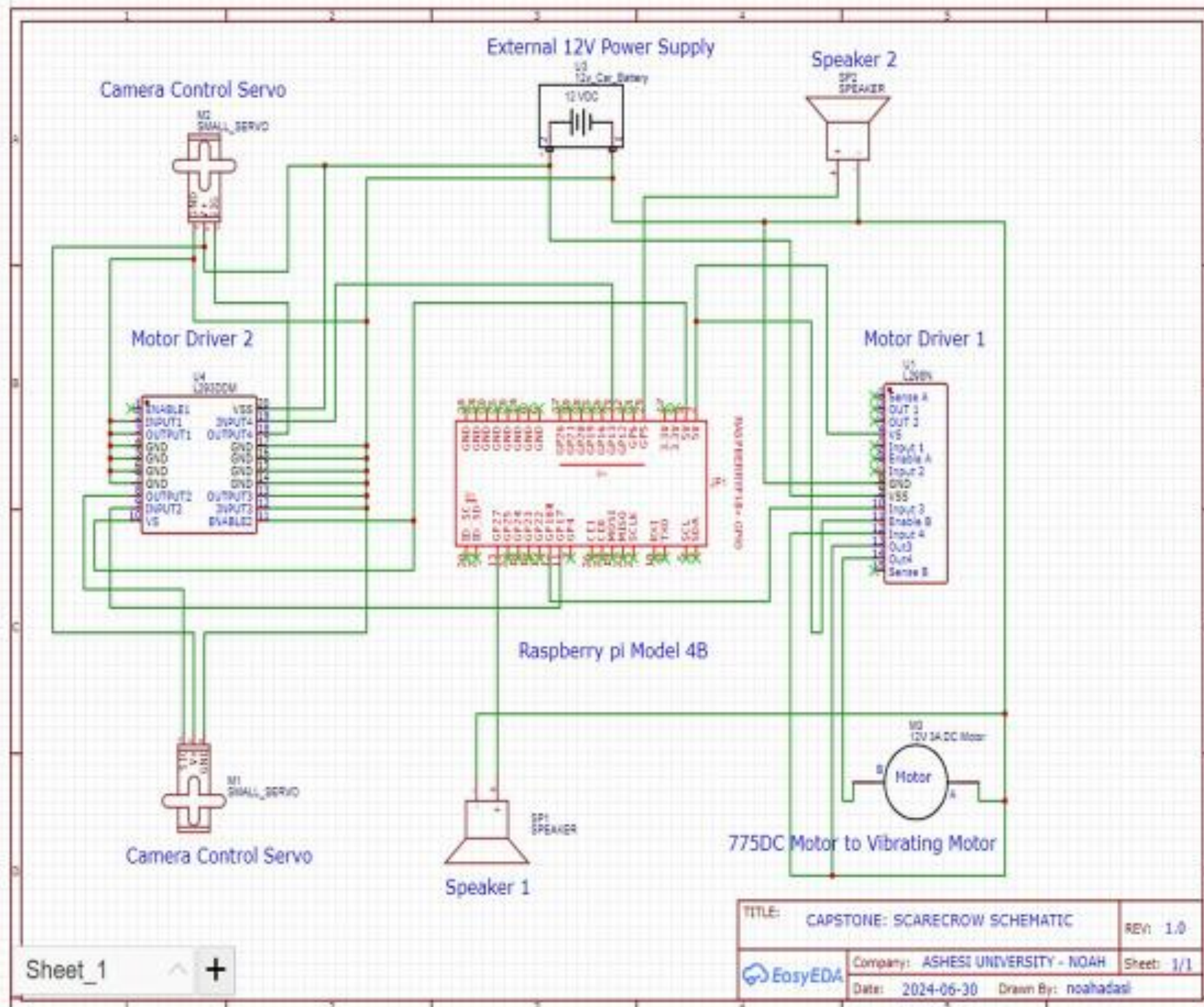
Solar Charge controller

Load

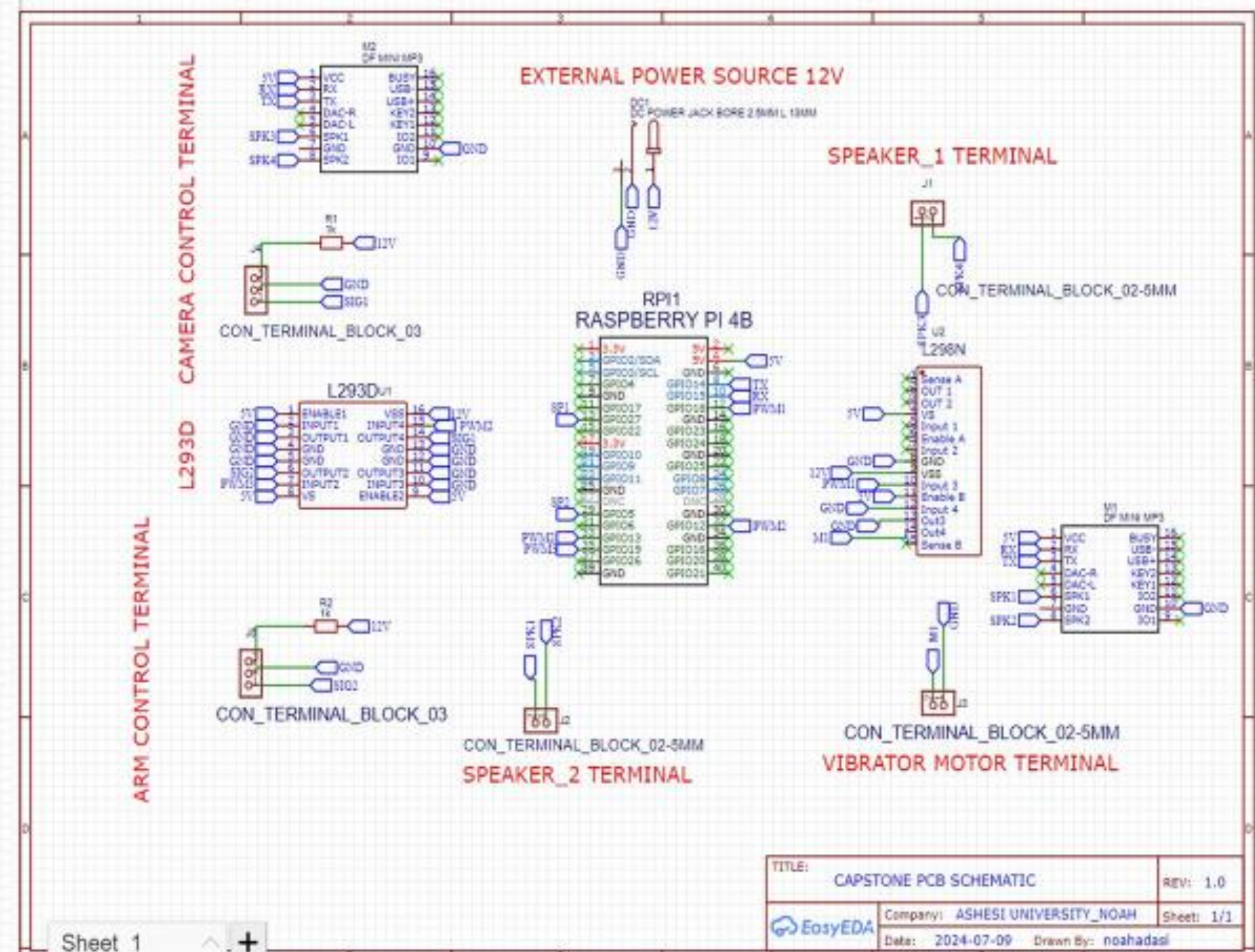
Electrical Schematics

LARANA, INC.

Conceptual schematic



PCB schematic



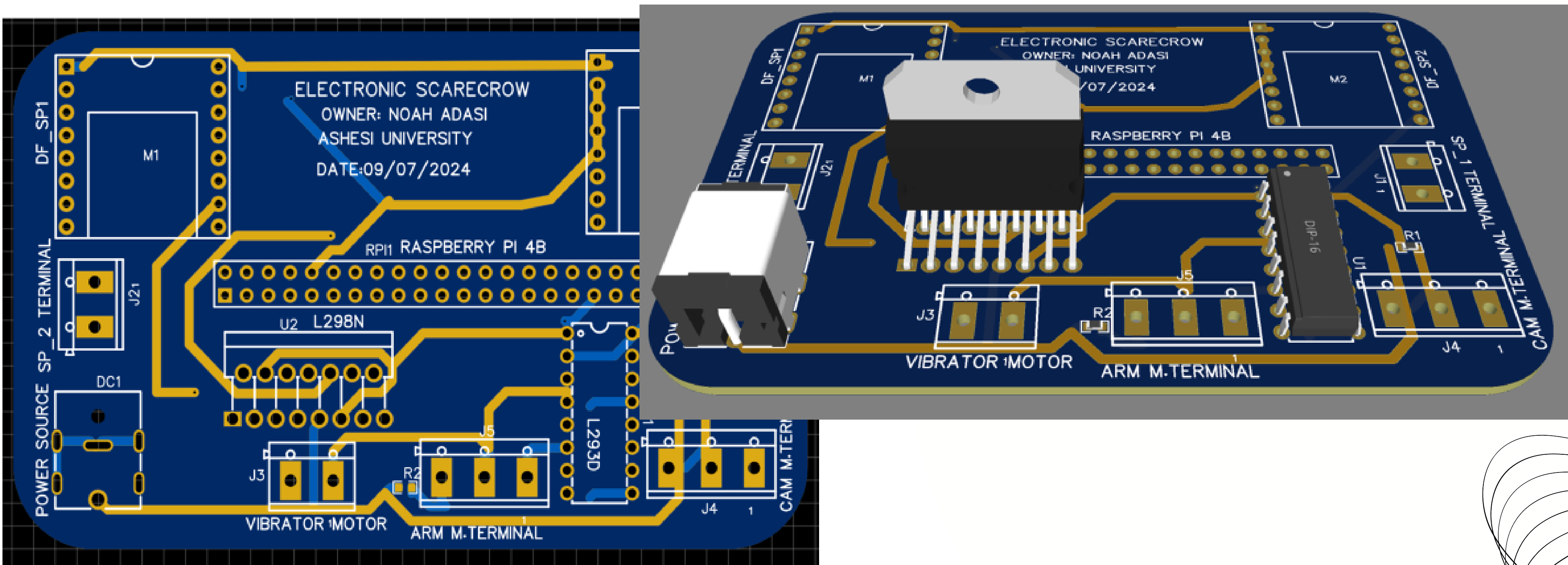
Electrical:

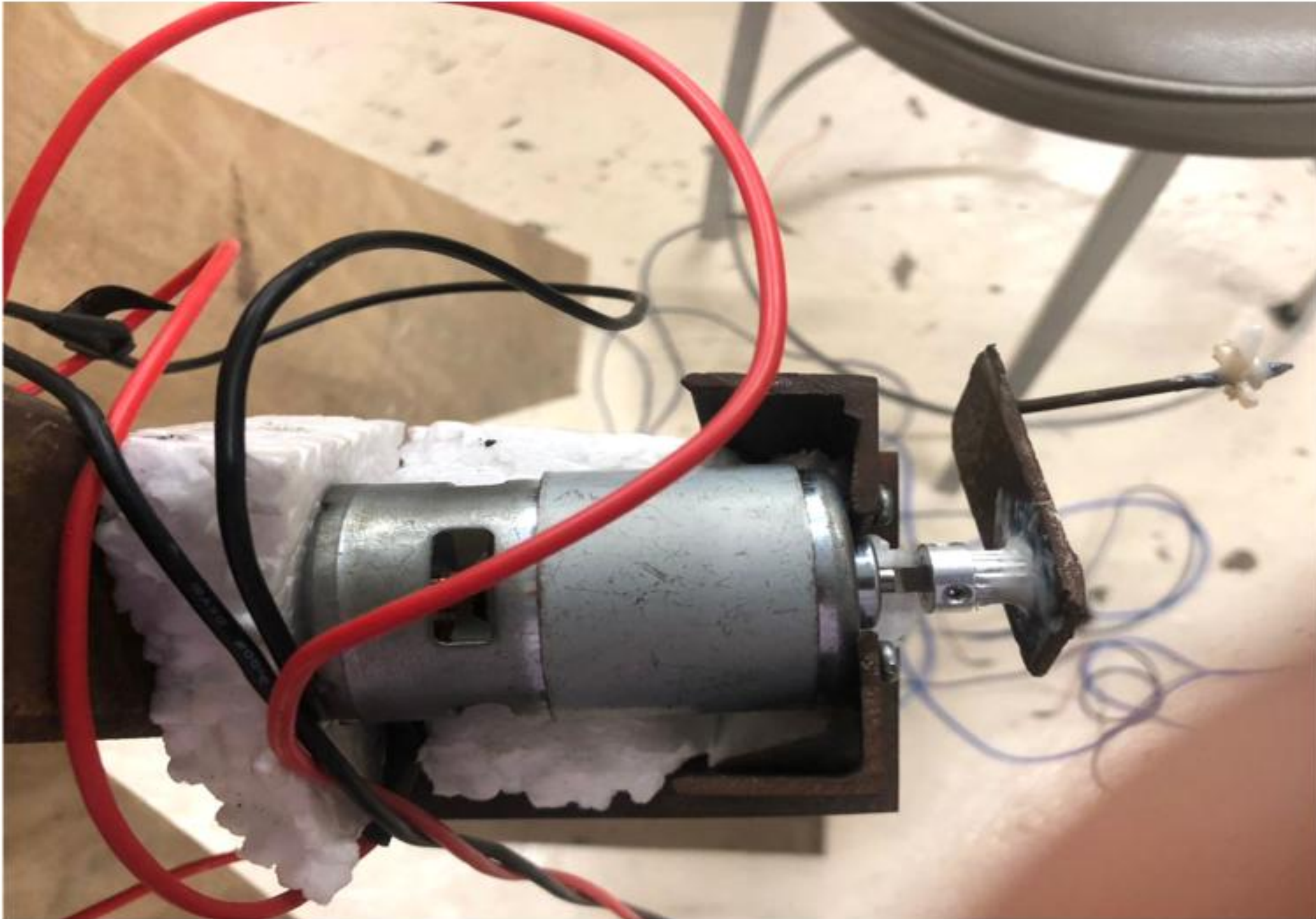
PCB

Realization

2D & 3D

LARANA, INC.





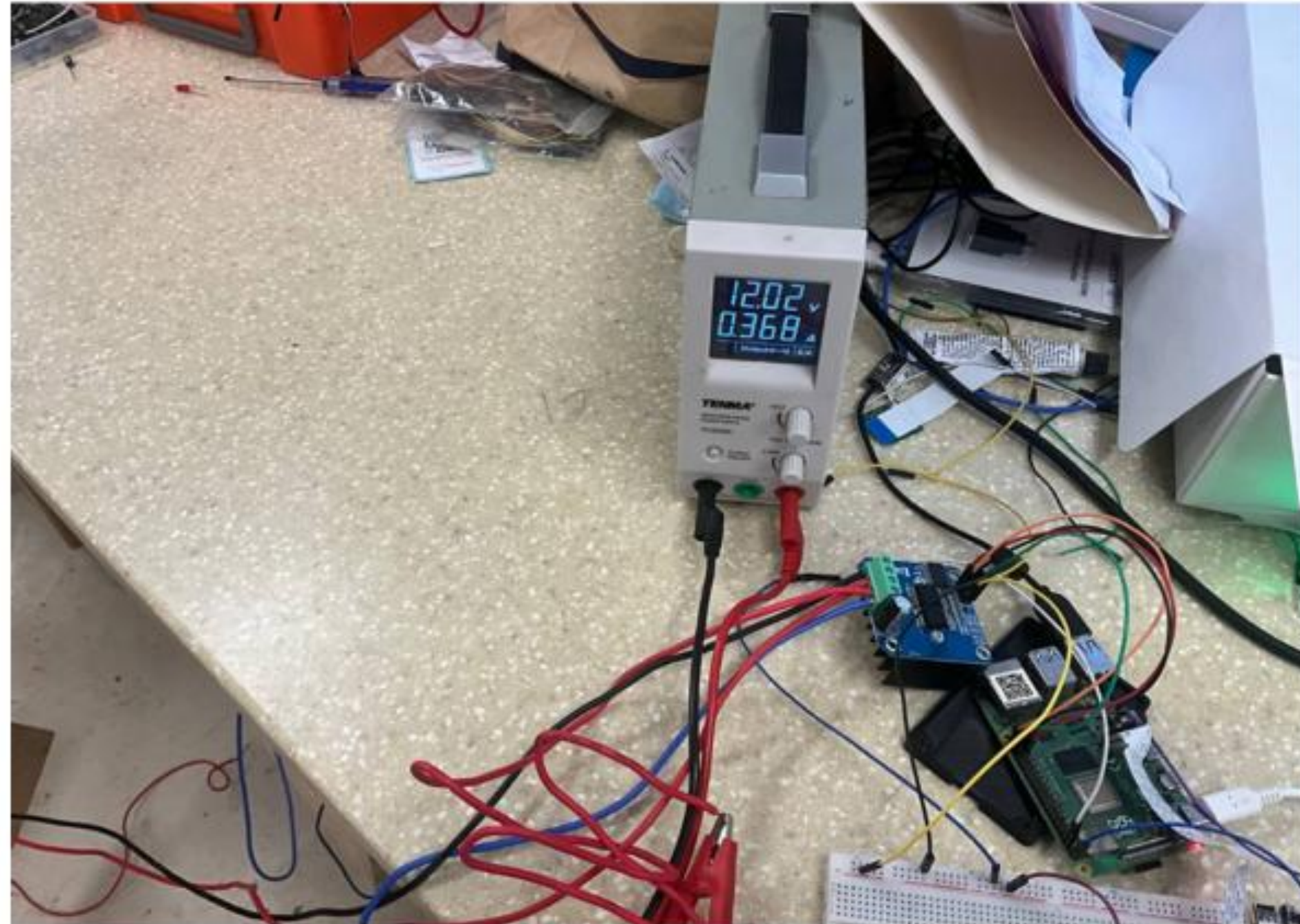
Vibrator motor

Parameters Considered

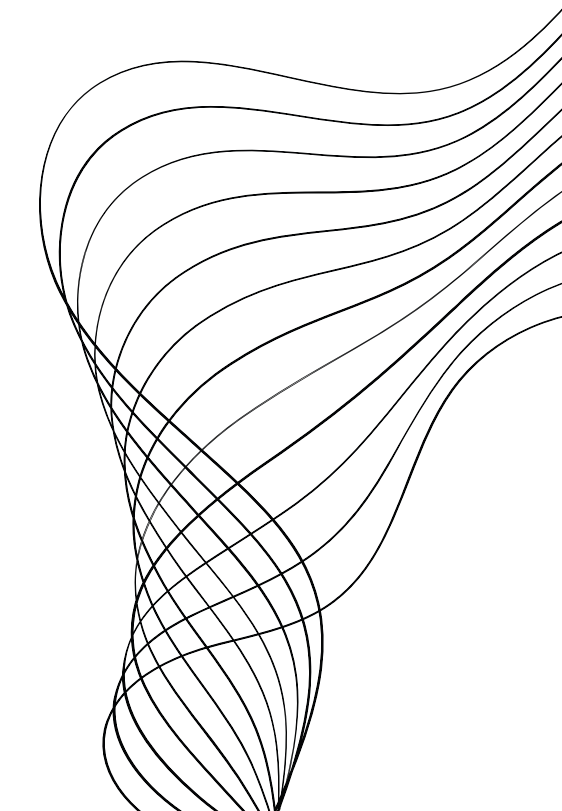
Impact of Mass on Vibration

- ❑ Vibration Intensity: A larger mass can increase the vibration intensity, but it also requires more torque to move

Vibrator motor



Training a YOLOv8 Model for Birds Detection



Introduction to YOLOv8

1 What is YOLOv8?

YOLOv8 stands for "You Only Look Once" version 8, an advanced real-time object detection model. It is known for its speed, accuracy, and efficiency, making it suitable for various applications.

2 Key Features

YOLOv8 boasts several key features, including its ability to process images in real-time with minimal latency, high precision in detecting and localizing multiple objects, and its design for performance on both high-end GPUs and edge devices.

3 Applications

YOLOv8 finds applications in diverse fields such as autonomous driving, surveillance, surveillance, retail, and healthcare, where real-time object detection is crucial.

crucia.



Dataset Preparation

Preprocessed Dataset
was used

Other details:

10000 birds' images
with 100 bird's
species

Single Image Dimension: 640 (height) x 640
(width) x 3 (channels) = 1,228,800 pixels

- Sourced from preprocessed publicly available datasets from Kaggle
- **Annotation Process:** Labelling, an open-source graphical image annotation tool. Each image was annotated with a corresponding .txt file containing YOLO format annotations, including class labels and bounding box coordinates.
- All images resized to 640x640 to ensure uniformity in input size.
- Channels: 3 (RGB)



Model Evaluation

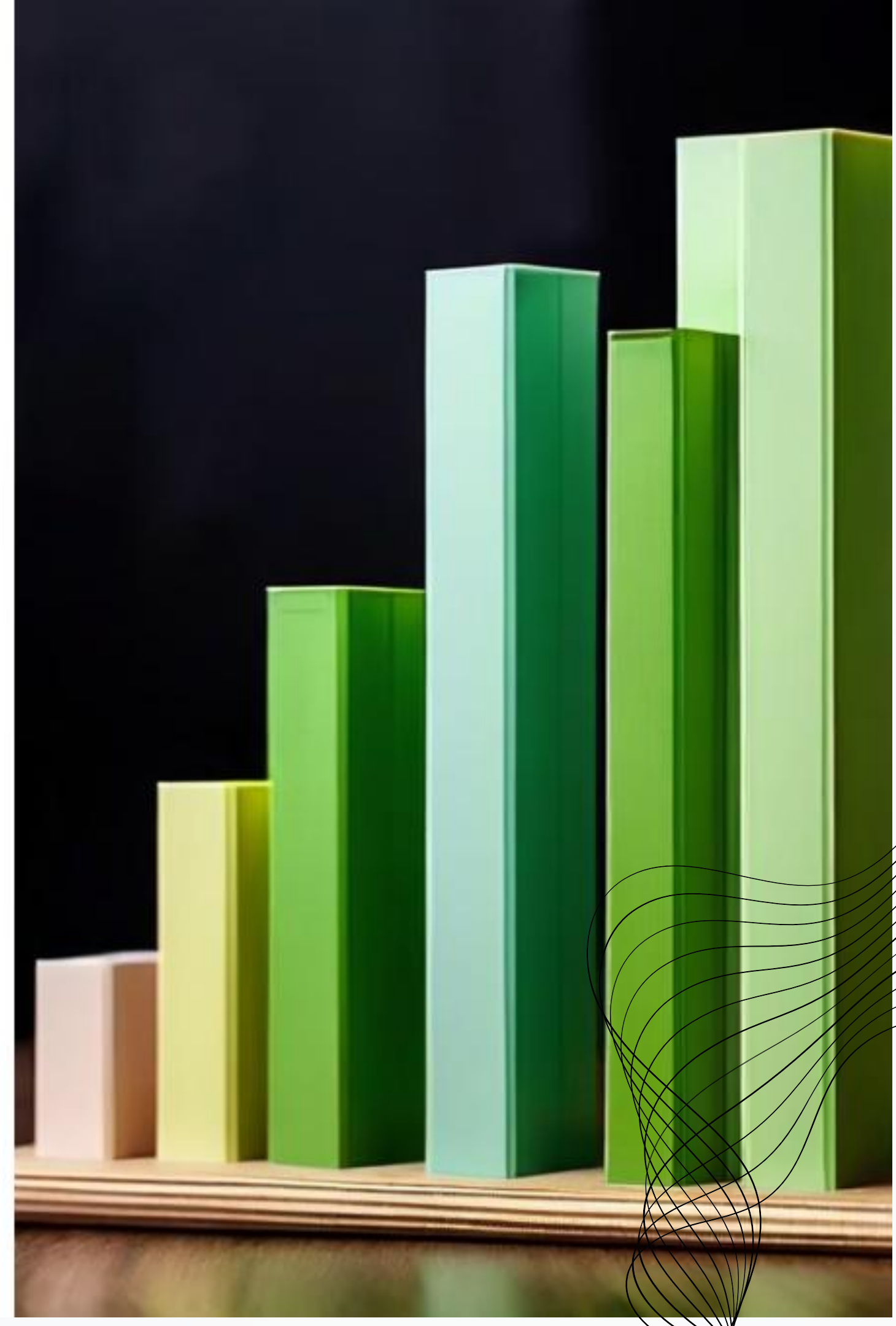


Evaluation Metrics

The yolov8's model's performance was evaluated using metrics like precision, recall, and mAP (mean Average Precision). These metrics provided insights into the model's accuracy and ability to correctly identify birds.



- Precision (98%): Out of all the birds detected by the model, 98% were correctly identified.
- Recall (94%): The model successfully detected 94% of all the actual birds present in the images.
- mAP (90%): The model achieved an average precision of 85% across all birds' classes and different levels of recall.





YoloV8



Training Parameters

Parameter	Value	Rationale
Epochs	100	Ensures sufficient training and prevents overfitting.
Batch Size	16	Balances computational load and convergence speed while considering memory constraints.
Image Size	640x640	Ensures compatibility with YOLOv8 architecture and maintains sufficient detail for accurate detection.

Learning rate: The initial Learning rate was set to 0.01 and then I decided to increase it by 0.1 in every 20 epoch



Hardware used & other considerations

Hardware Setup

The training was conducted on Google Colab, leveraging the NVIDIA Tesla T4 GPU for accelerated computations. The Colab environment provided a standard multi-core CPU and approximately 12-16GB RAM.

Runtime

The training process took approximately 12 hours for 100 epochs using.

1

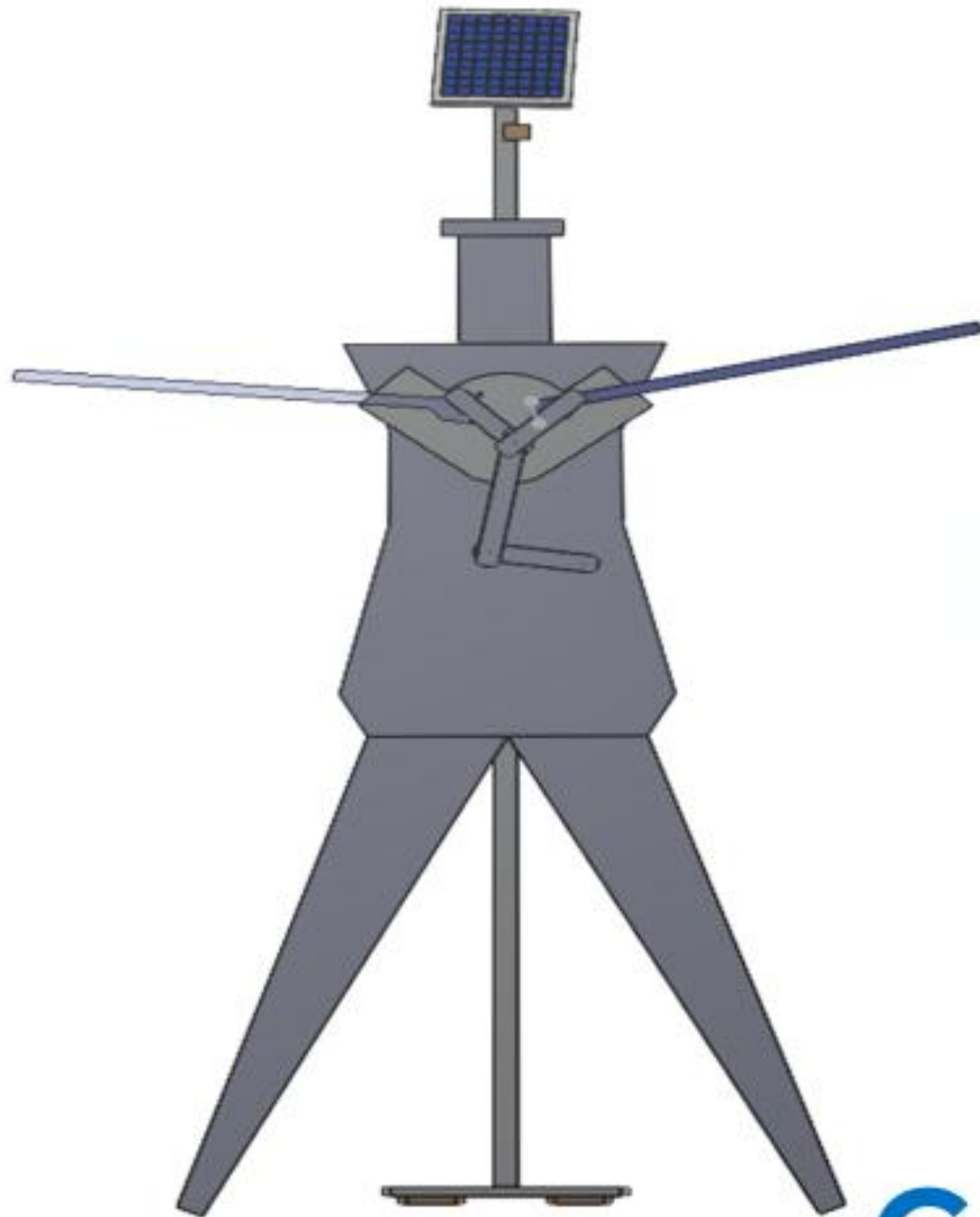
2

3

Training Environment

The Ultralytics YOLOv8 library with PyTorch backend was used for training. Challenges included memory management, adjusting batch size and image size to fit within GPU memory,

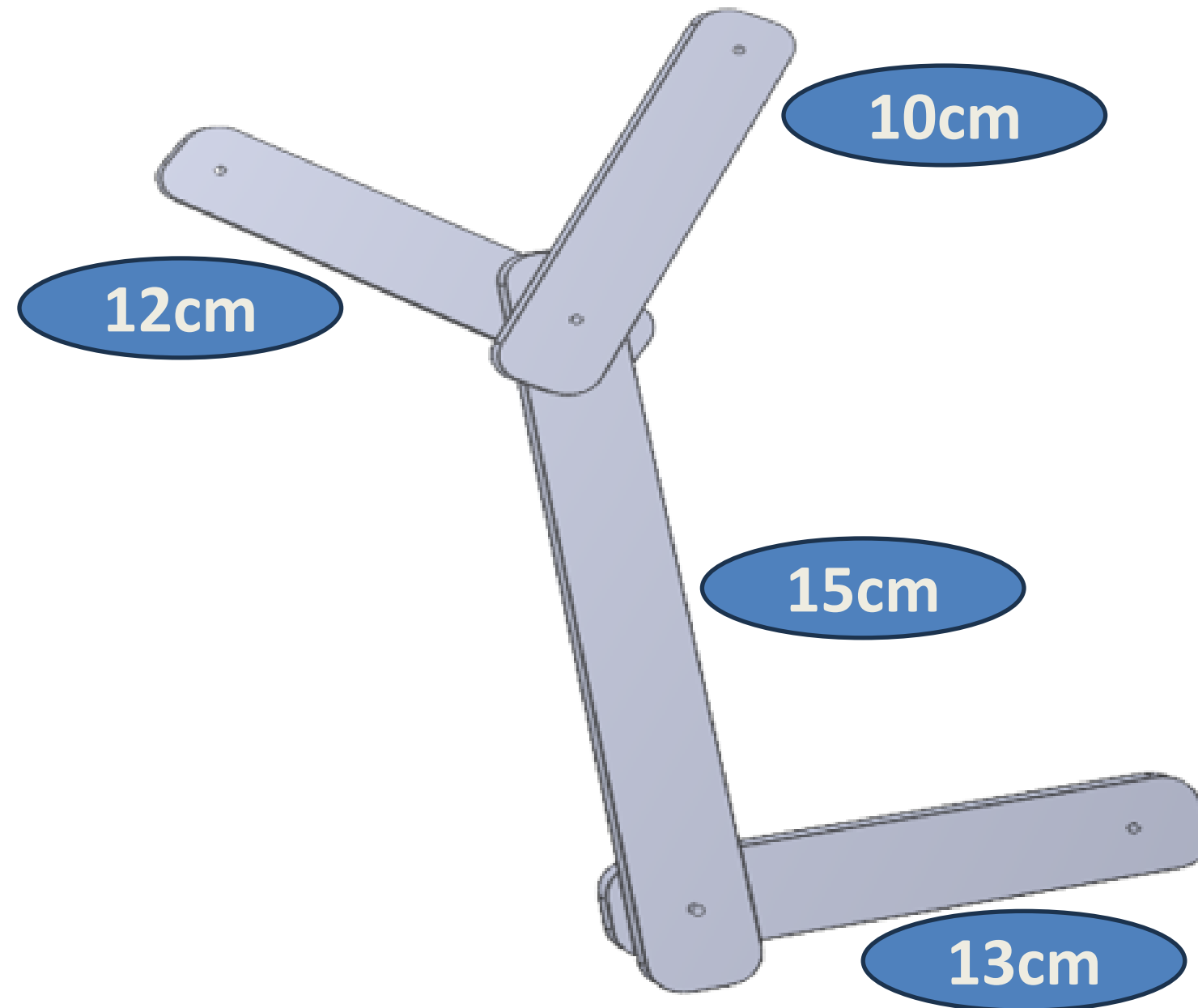
Mechanical Design



CAD Model



Mechanical Design



Four arm linkage

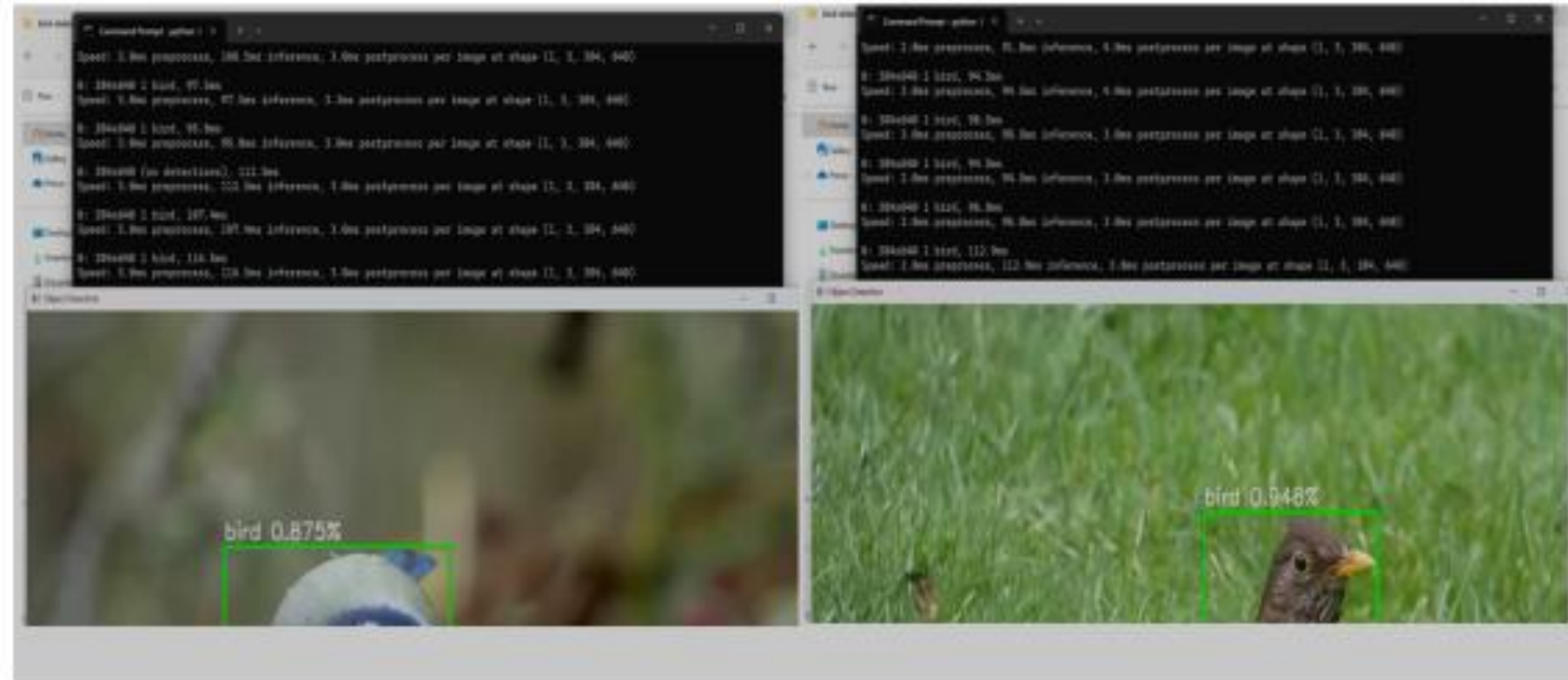
$$S+L \leq P+Q$$

Implementation of the System



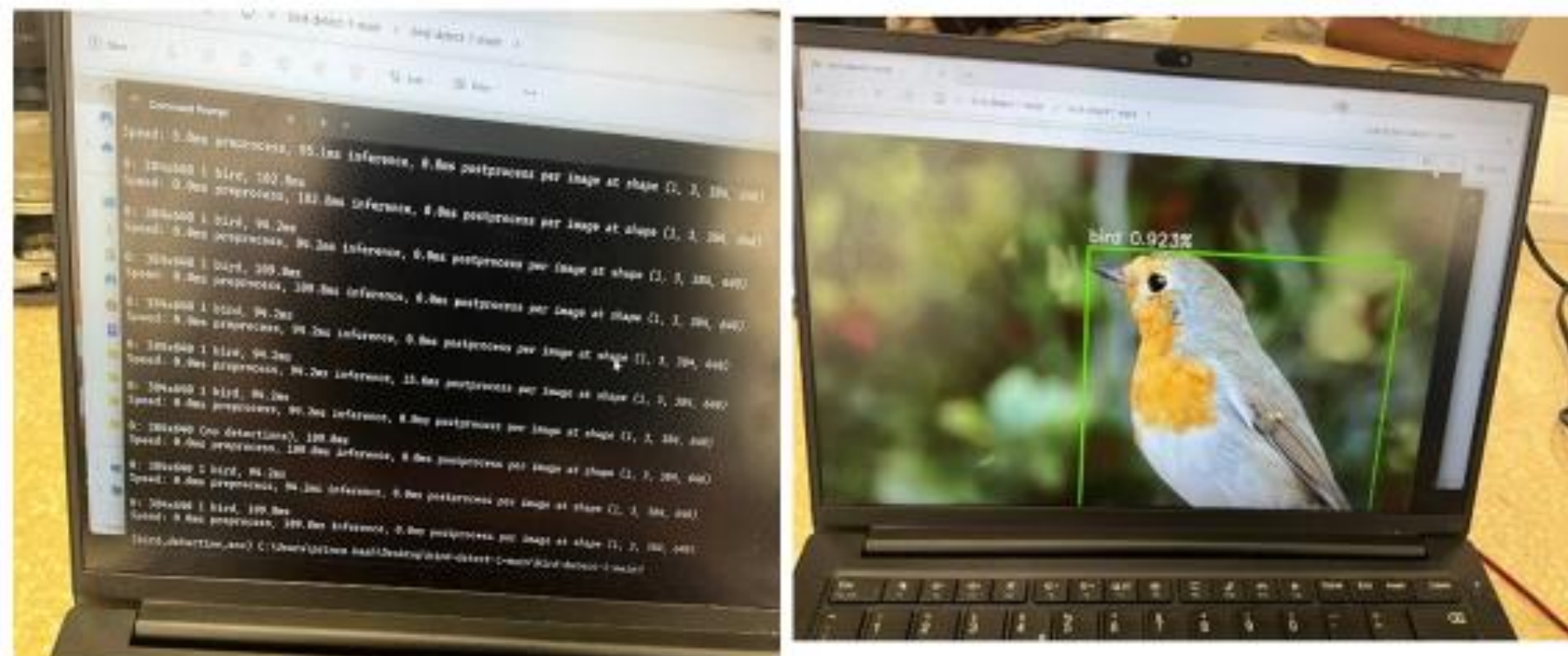
Testing, Results & Analysis

With Video Frame



Average inference time of 3ms per frame and a confidence threshold of 0.8. Processing time: 94 ms, speed 2ms

with the laptop camera



Testing, Results & Analysis

Raspberry Pi and video frames



```
File Edit Tabs Help
geany_run_script_ZD2A92.sh

Helper for embedded data usage!
[0:02:35.765839000] [2219] INFO RPI vc4.cpp:446 Registered camera /base/soc/12c
0mux/12c01/imx219010 to Unicam device /dev/media4 and ISP device /dev/media0
[0:02:35.766052421] [2219] INFO RPI pipeline_base.cpp:1104 Using configuration
file '/usr/share/libcamera/pipeline/rpi/vc4/rpi_apps.yaml'
[0:02:35.785683158] [2189] INFO Camera camera.cpp:1183 configuring streams: (0)
1280x720-X86R8888 (1) 1920x1080-SBGGR10_CSI2P
[0:02:35.767285561] [2219] INFO RPI vc4.cpp:621 Sensor: /base/soc/12c0mux/12c01
/imx219010 - Selected sensor format: 1920x1080-SBGGR10_1X10 - Selected unicam fo
rmat: 1920x1080-pBAA

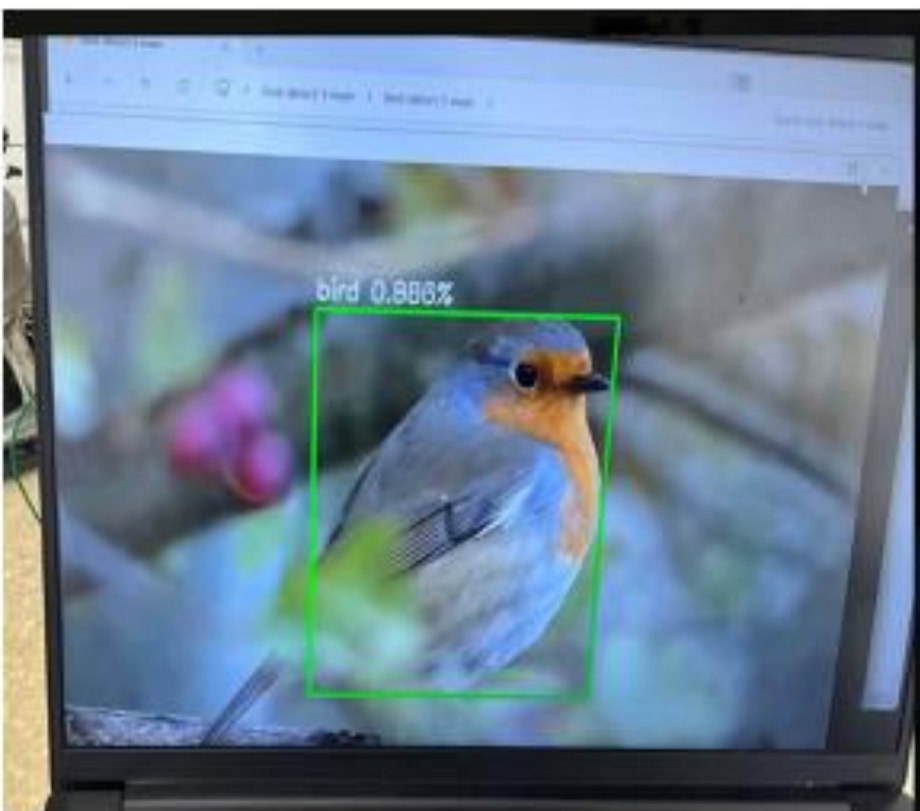
0: 384x640 (no detections), 1614.0ms
Speed: 22.2ms preprocess, 1614.0ms inference, 33.3ms postprocess per image at sha
pe (1, 3, 384, 640)

0: 384x640 (no detections), 2544.1ms
Speed: 12.1ms preprocess, 2544.1ms inference, 5.5ms postprocess per image at sha
pe (1, 3, 384, 640)

0: 384x640 (no detections), 1672.8ms
Speed: 19.0ms preprocess, 1672.8ms inference, 3.0ms postprocess per image at sha
pe (1, 3, 384, 640)
```

**Average Processing time is 1.8 s &
Average inference time of 13.6 milli
seconds per frame**

Raspberry Pi with picamera



Testing, Results & Analysis

- ❑ Testing of the Vibrator Motor Was done



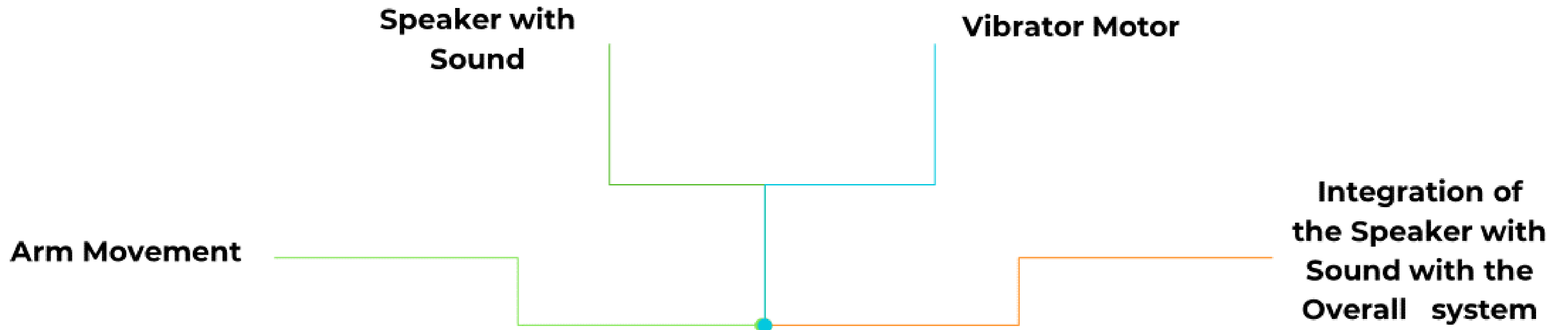
- ❑ Testing of the Speaker with the sound was done



- ❑ Testing of the arm movement was also done



Demonstration



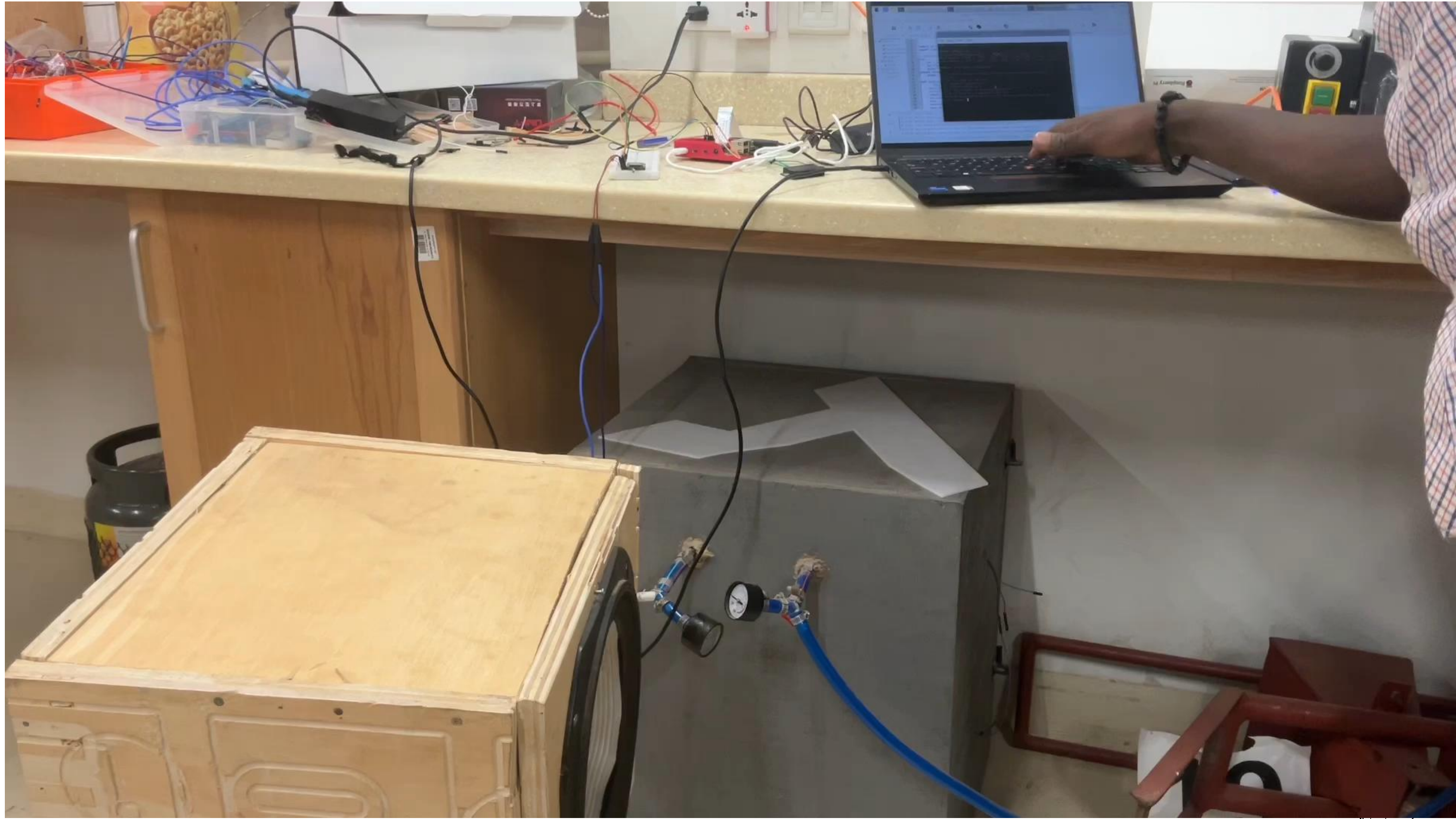
Demonstration

Arm Movement



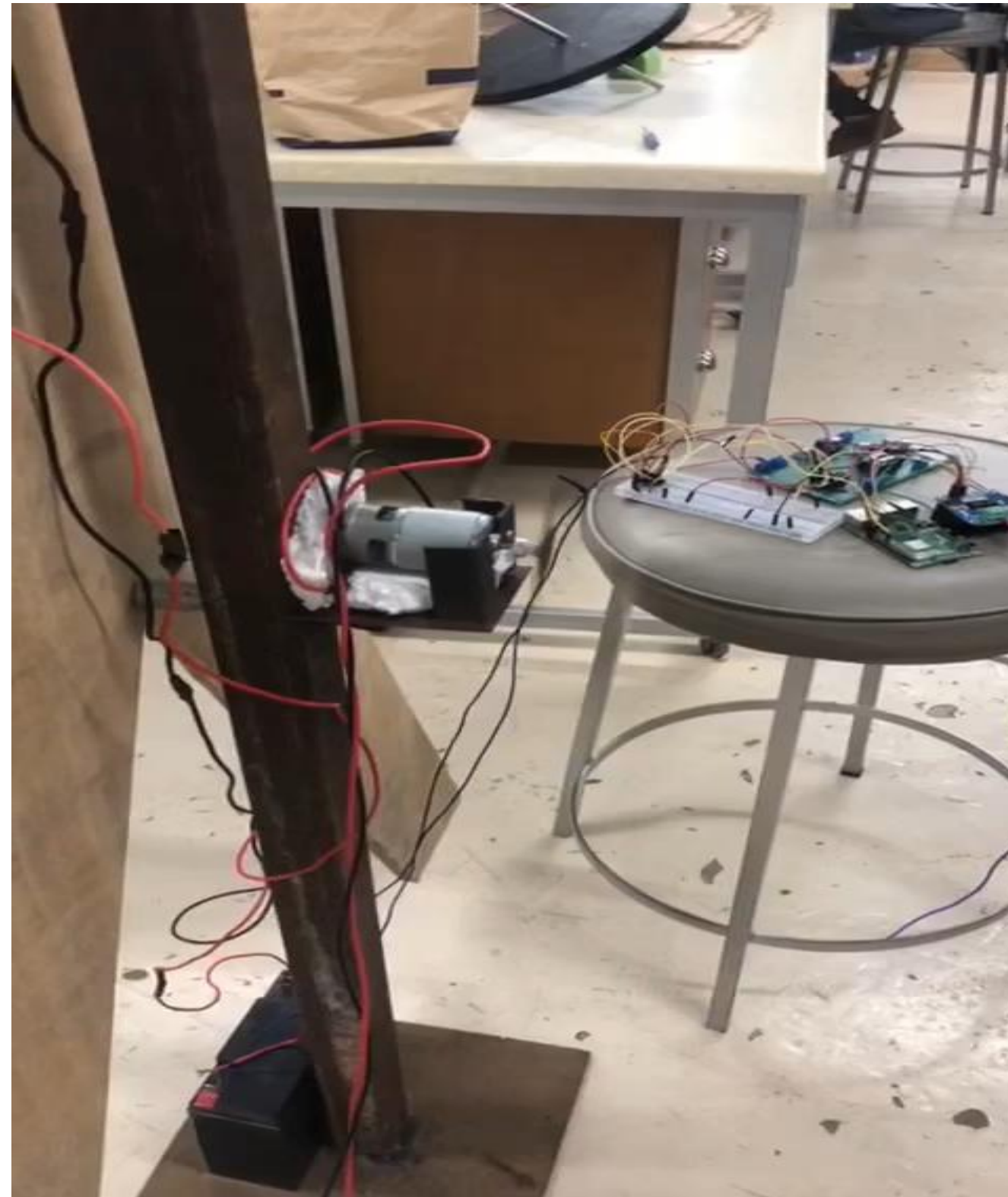
Demonstration

**Speaker with
Sound**



Demonstration

Vibrator Motor

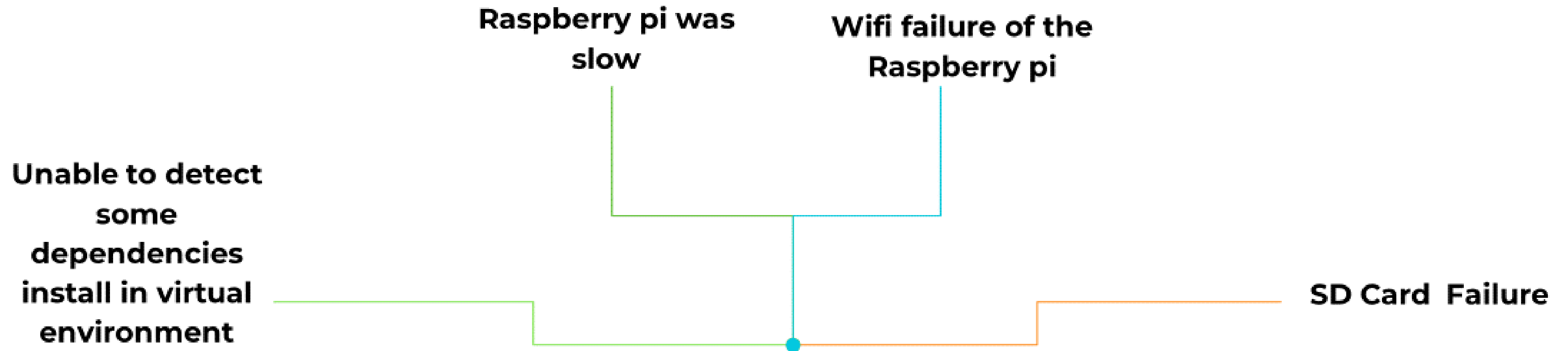


Demonstration

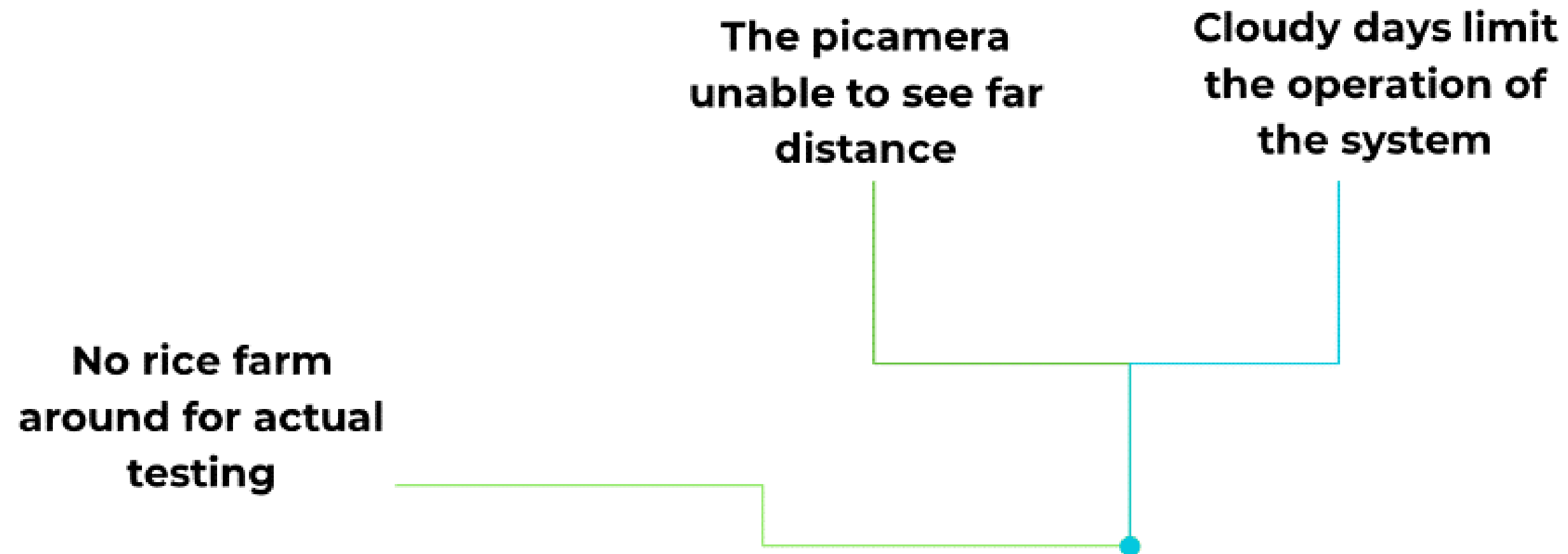
Integration of
the Speaker with
Sound with the
Overall system



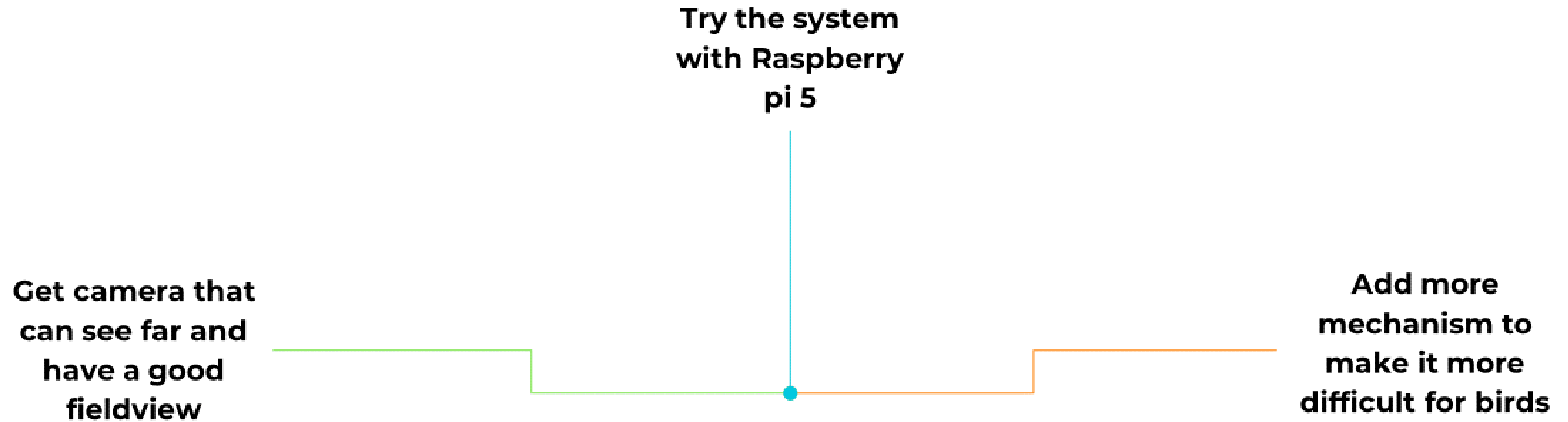
Challenges faced

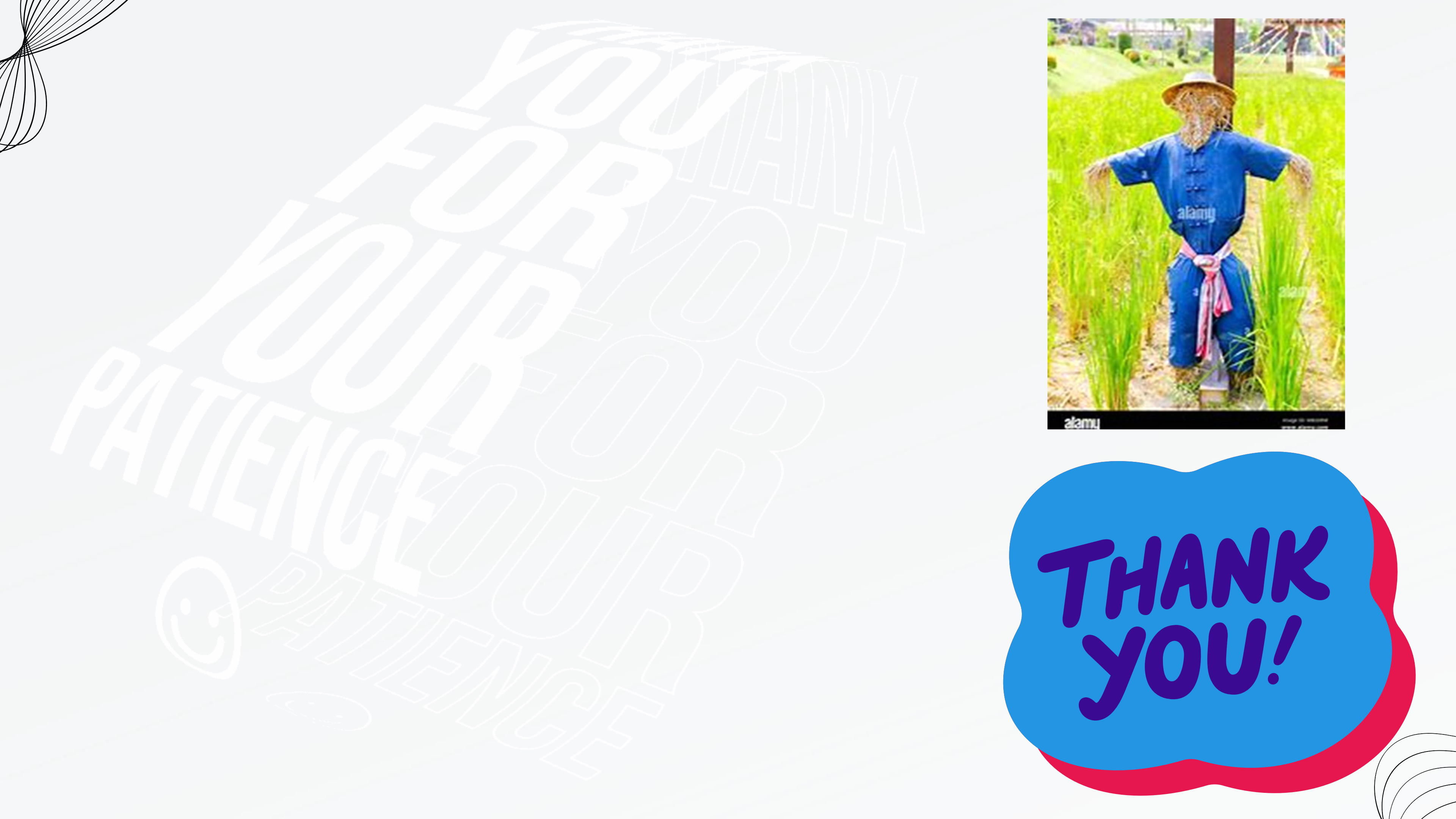


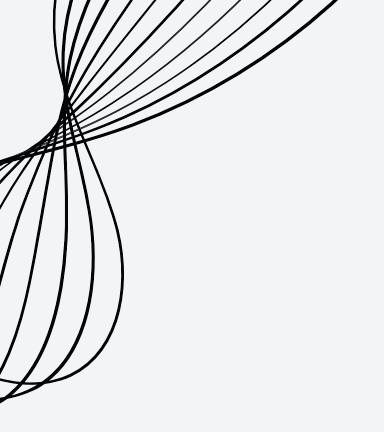
Limitations



Future Works

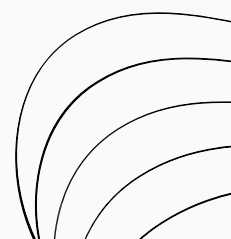






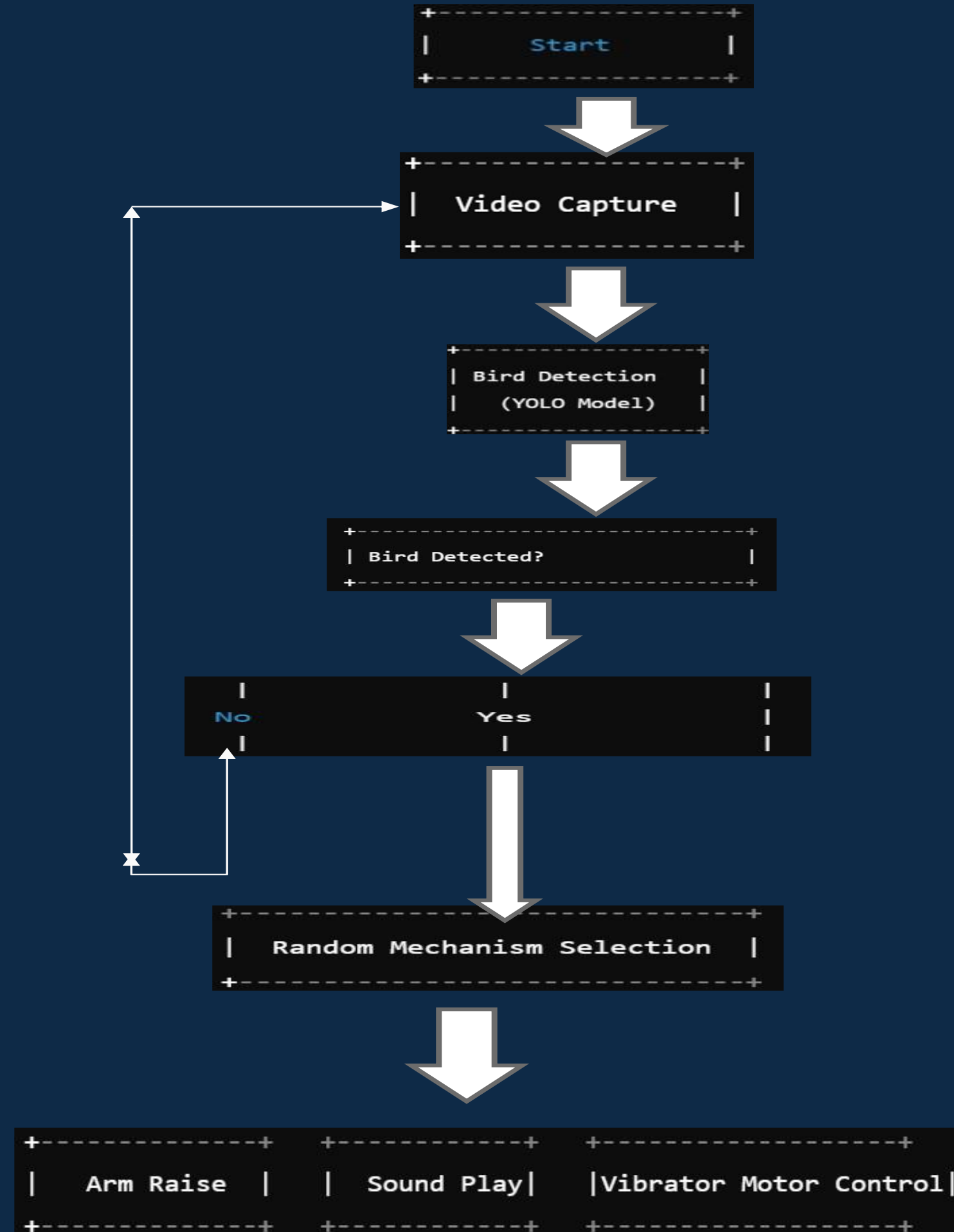
Appendix

[\(255\) The Human Scarecrows of Akuse \(How to scare birds in rice farms\) – YouTube](#)





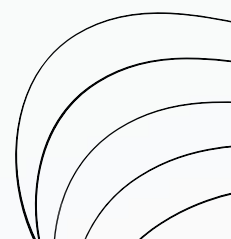
HOW SHIELD YOUR FARM WORKS –FLOW DIAGRAM

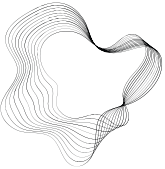


HOW SHIELD YOUR FARM WORKS –FLOW DIAGRAM



Achieved objectives

- ✓ **Two dimensional arm movement mechanism**
 - ✓ **One Traditional Approach Automation**
 - ✓ **Using Sound Mechanism**
 - ✓ **Leveraging on Machine Learning for bird detection**
- 



Selecting Suitable Power Source

$$PT = [14.4 + 15 + 14.4 + 0.1152 + 3] W$$

$$PT = 46.9152W$$

Battery Specifications:

$$Voltage = 12V$$

$$Capacity = 17Ah$$

Assumptions made

Assuming continuous operation

- Camera: (P_camera) - Servo: (P_servo)

Occasional Components:

- Sound and Arm activation - Vibrating Motor

- Total power for occasional components: (P_occ = 46.9152W -P_cont)

Power Requirement for Continuous Operation:

To run the scarecrow for 9 to 10 hours:

We calculate the continuous current draw:

$$I_{cont} = P_{cont}/V$$

Given the battery capacity of 17Ah:

$$Runtime = Capacity/I_{cont}$$

To ensure the scarecrow runs for 9 to 10 hours, we need:

$$I_{cont} = 17Ah/10hours = 1.7A$$

LARANA, INC.

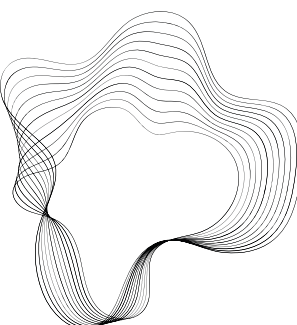
Power Consumption of Camera and Servo:

Assume:

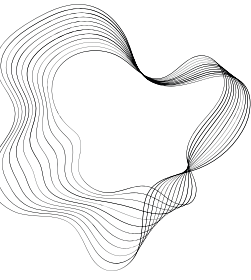
Camera: 5W

Servo: 5W

$$P_{cont} = 5W + 5W = 10W$$



Selecting Suitable Power Source



LARANA, INC.

$$I_{cont} = 10W/12V = 0.833A$$

Runtime:

$$Runtime = 17/0.833 = 20.4hours$$

This is sufficient for continuous operation. So, if we account for occasional components:

Assume occasional operation for 1 hour of total active time per day:

$$Total\ occasional\ power: = 46.9152W - 10W = 36.9152W$$

$$I_{occ} = 36.9152W/12V = 3.076A$$

$$Q_{occ} = 3.076A * 1hour = 3.076Ah$$

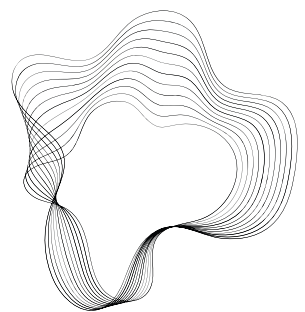
Combined Runtime:

Total current draw for continuous operation over 9 hours:

$$\begin{aligned} I_{total} &= 0.833A * 9hours + 3.076Ah \\ &= 7.497Ah + 3.076Ah \\ &= 10.573Ah \end{aligned}$$

The battery can support this this Scarecrow, providing:

$$Total\ runtime = 17Ah/10.573Ah/day = 1.6\ days$$



Position Analysis



- **Input Link (a):** The link that is driven by the input motion.
- **Coupler Link (b):** The link that connects the input and output links and often moves in a complex path.
- **Follower Link (c):** The link that is driven by the coupler link and provides the output motion.
- **Fixed Link (d):** The stationary frame or base of the mechanism.

- θ_1 : Angle of the input link relative to a reference frame
- θ_2 : Angle of the coupler link relative to a reference frame
- θ_3 : Angle of the follower link relative to a reference frame

Equation for Sines:

$$a \sin(\theta_1) + b \sin(\theta_2) = c \sin(\theta_3)$$

Equation for Cosines:

$$a \cos(\theta_1) + b \cos(\theta_2) = d + c \cos(\theta_3)$$

Velocity Analysis:

Differentiate the position equations with respect to time:

$$\frac{d}{dt} (a \cos(\theta_1) + b \cos(\theta_2)) = \frac{d}{dt} (d + c \cos(\theta_3))$$

$$-a\dot{\theta}_1 \sin(\theta_1) - b\dot{\theta}_2 \sin(\theta_2) = -c\dot{\theta}_3 \sin(\theta_3)$$

where $\dot{\theta}_i$ represents the angular velocity of link i.

Acceleration Analysis:

Differentiate the velocity equations with respect to time:

$$\frac{d}{dt} (-a\dot{\theta}_1 \sin(\theta_1) - b\dot{\theta}_2 \sin(\theta_2)) = \frac{d}{dt} (-c\dot{\theta}_3 \sin(\theta_3))$$

$$-a(\ddot{\theta}_1 \sin(\theta_1) + \dot{\theta}_1^2 \cos(\theta_1)) - b(\ddot{\theta}_2 \sin(\theta_2) + \dot{\theta}_2^2 \cos(\theta_2)) = -c(\ddot{\theta}_3 \sin(\theta_3) + \dot{\theta}_3^2 \cos(\theta_3))$$

where $\ddot{\theta}_i$ represents the angular acceleration of link i.